

Haute École Arc - Ingénierie

Panique en Parapente

ISC3id-a, 2025- 2026

Dans le cadre du P3 au cours du semestre d'automne

Alex Houttuin

Superviseur: Dr. Hatem Ghorbel

Proposé par: Cédric Bilat

January 23, 2026

Alex Houttuin

Abstract

Paragliding in Switzerland is challenging and dangerous, where judging terrain clearance by eye can be difficult. The "Panique en Parapente" project aims to improve pilot safety by providing a real-time decision tool directly on a mobile device.

This application calculates the exact altitude difference required to clear obstacles between the pilot and a destination, and gives the altitude at the destination after clearing these obstacles. By utilizing high-resolution LiDAR data from SwissTopo and ensuring strictly offline functionality, the tool offers guidance even in irregular terrain. A key component of the program's architecture is the implementation of a dynamic tile controller, which allows the application to process the voluminous topographical datasets within the limited memory constraints of a mobile device.

The results demonstrate that the system is both fast and precise, capable of detecting small obstacles like walls in near-real-time. The application also integrates audio feedback, to enable hands-free operation and keep the pilot focused on the surroundings. While the current implementation focuses on vertical clearance within Switzerland, the established framework offers a foundation for future integration of lateral pathfinding and global terrain data.

Contents

1	Introduction	3
2	Analysis	3
2.1	Primary objectives	4
2.2	Secondary objectives	4
2.3	Nice-to-have	5
2.4	Out of scope	5
3	Conception	6
3.1	Architecture	6
3.2	Topography	6
3.3	1D Algorithm	8
3.4	GPS	9
4	Implementation	10
4.1	Technologies used	10
4.2	Data optimisation	10
4.3	Parallel downloading	10
4.4	GPS altitude calibration	11
4.5	User interface	11
5	Results and discussion	12
5.1	Topography	12
5.1.1	Tile download time	12
5.1.2	Tile visualisation	13
5.2	1D algorithm	15
5.2.1	Unit tests	15
5.2.2	Algorithmic time	15
5.2.3	Tile controller	16
6	Conclusion	19
6.1	Going further	19
6.1.1	Fast marching trees	19
6.1.2	Hardware integration	20
7	Sources	20

1 Introduction

In 2009, an airplane took off from LaGuardia Airport in New York City. A flock of birds caused a double engine failure and the pilots needed to find a way to glide to safety. Not knowing where would be safest, they landed on water by trusting their instinct, managing to keep all 155 people on board alive, although most were injured.

Just as there was no tool for those pilots to determine where to land in case of emergencies, so too are there no tools for paragliders specifically designed for real-time terrain clearance calculations, to help guide them safely. In Switzerland, there are numerous cases of injuries every year from paragliding, and some accidents can even be fatal [1]. A pilot often needs to assess if they can reach a certain point, based on their altitude, their glide ratio and the distance and height of that point, but they lack the predictive tools that integrate real-time terrain data in order to confirm clearance.

This project, titled "Panique en Parapente", was conducted during the P3 course at the Haute École Arc Ingénierie. It provides a tool on mobile devices that assists paragliders in gauging the altitude required to safely reach a designated waypoint. The system uses SwissTopo's data for high-resolution topological data in order to calculate terrain clearance in near-real-time.

The report is structured as follows: We will begin with an analysis of the problem and the specific client requirements. Next, we will present the conception of the solution for it, including mathematical models and architectural choices. Finally, we will detail the implementation the solution and then we will conclude by discussing the results obtained during testing and analyzing.

2 Analysis

The goal of this project is to design and develop a tool that would assist a paraglider pilot to reach a designated waypoint, in real time, for mobile devices.

2.1 Primary objectives

1D algorithm

Our most important objective is to develop an algorithm that calculates the difference in altitude between the user position (GPS) and the waypoint, by taking into account the distance separating both, the terrain obstacles (derived from swissSURFACE3D Raster data [2]) and the paraglider’s fineness ratio (the pilot travels a distance of f for every meter of altitude dropped). It should return a tuple containing both the minimum climb needed Δh_{climb} to reach the destination, and the altitude at destination h_{arrival} , which is important for landing preparation. The algorithm needs to be both fast and precise, and it has to provide an update every 10 seconds so that the pilot has near-real-time data.

Interface

We want to provide an abstract interface permitting easier future implementations of different providers. For the sake of this project, we should implement both a real time provider and a simulated one.

Offline

To minimize dependence on network connectivity, we will ensure a minimum of network dependency during flight except for periodic user position updates. All data must therefore be cached prior to flight time.

Mobile

The tool should work as is on mobile devices, and we should consider their performance limitations when implementing our tool.

2.2 Secondary objectives

Voice

As the user should focus their attention on where they are going, we need to provide a text-to-speech feature to allow hands-free obtention of information.

Visualization

We work with data provided by an API, so to ensure the data corresponds to reality, we visualize it and compare it.

2.3 Nice-to-have

2D algorithm

If time permits it, we would like to upgrade the primary algorithm to enable path finding for the pilot. It would give the pilot a path to follow in addition to the previous climb and waypoint altitude information.

Global topology

We would like to see if we can extend our tool to cover other regions outside of Switzerland.

2.4 Out of scope

These points were evaluated and eventually considered as out-of-scope.

Meteorological data

Getting information on wind, temperature, humidity, and much more, and implementing them in a way that affects our primary algorithm would make the tool more realistic and thus safer. This comes with a high cost in time to implement and performance limitations on mobile devices.

HPC offloading

In case calculations were to be too intense for mobile devices, offloading them to a high powered computer server and getting back solely the output is a possibility. However, this approach requires an always-on server and reliable connectivity, which goes against our primary objective to be as offline as possible.

3 Conception

3.1 Architecture

Program controller

The program controller ensures that the tool runs fluidly. It synchronizes the user's GPS position, the tile controller for tile retrieval and the primary algorithm within an update loop (a tick) in a 10 second interval.

Service elevation

An abstract interface `ServiceElevation` was designed to make it easier to switch between real data (from SwissTopo) and a simulated terrain for unit tests.

3.2 Topography

Data source

For the surface elevation data, we chose **swissSURFACE3D Raster**, which is a Swiss-wide grid of rasterized GeoTIFF images of LiDAR points captured from multiple angles encompassing not only the surface, but buildings and forests as well. We retrieve *tiles* via a STAC API [3], which are each 1 km² and have a resolution of 0.5 meters between pixels. We feed the STAC API with a bounding box calculated from the user-given radius around the waypoint, which will return all tiles that are within the bounding box.

Coordinate systems

WGS84 (latitude and longitude) and LV95 (northing and easting) are both systems of coordinates we employ. WGS84 works by creating an ellipsoidal representation of the Earth, while LV95 is a cartesian projection (x and y) in meters. We will convert between the two of them as our device works in the WGS84 coordinate system (getting GPS data and calculating Haversine distance) while our tiles are indexed using simplified LV95 coordinates in kilometers (ex: 2560-1205, instead of 2560000-1205000).

Earth curvature

The bounding box for the STAC API is comprised of a minimum and maximum latitude and longitude, composing of 4 coordinates that are relative to the waypoint in the middle. These coordinates are calculated from the radius r provided by the user.

Due to the convergence of the meridians, as we approach the poles [4], the distance between degrees of longitude decreases, while on a Mercator map these distances are parallel. That's why a bounding box in Norway will not look the same as one in Kenya for example. To make sure a bounding box is a perfect square, we will need to compensate this with calculations:

$$\Delta_{\text{lat}} = \frac{r_{\text{km}}}{111.32}$$
$$\Delta_{\text{lon}} = \frac{r_{\text{km}}}{111.32 \times \cos(\phi)}$$

With ϕ the latitude in radians:

$$\phi = \text{lat} \times \frac{\pi}{180}$$

And 111.32 the mean distance per degrees of latitude:

$$\frac{40'075}{360} = 111.32\text{km}$$

At the equator, longitudinal lines are equidistant with latitudinal lines, where a degree corresponds to ~ 110.57 km. At the poles, a degree is ~ 111.70 km. The mean is at 111.32 km. The discrepancy means we need to compensate the convergence to the poles with the cosinus function for longitudinal lines.

The bounding box should be approximate to the latitude and longitude of the center of the circle drawn by the radius r given by the user. With Δ_{lat} and Δ_{lon} , we can measure the bounds as such:

- $\text{min}_{\text{lat}} = \text{center}_{\text{lat}} - \Delta_{\text{lat}}$
- $\text{max}_{\text{lat}} = \text{center}_{\text{lat}} + \Delta_{\text{lat}}$
- $\text{min}_{\text{lon}} = \text{center}_{\text{lon}} - \Delta_{\text{lon}}$
- $\text{max}_{\text{lon}} = \text{center}_{\text{lon}} + \Delta_{\text{lon}}$

3.3 1D Algorithm

The primary algorithm should run in 10 second ticks, which should give ample time for calculation and give breathing room to the user receiving climb information.

Naive algorithm

The naive algorithm will simply return the difference in altitude needed to climb to reach the waypoint, without taking into account obstacles in terrain. It measures the drop in altitude relative to the fitness ratio for the distance to the waypoint, and adjusts it in the correct altitude relative to the user and waypoint.

$$\Delta h_{\text{climb}} = \frac{d}{f} + h_{\text{waypoint}} - h_{\text{user}}$$

Where f is the fitness ratio, d is the Haversine distance (see Annex A or [5] for more details) between the user and the waypoint, and h is the height relative to the terrain.

Obstacle-aware algorithm

In order to take into account any obstacle between the user and the waypoint, we must divide the segment into points. The naive algorithm runs for each of these points and returns the climb delta iteratively, keeping the lowest value as that indicates that the point is located under the terrain. After the algorithm has ran across all points, we take the absolute value of the lowest value to indicate the largest climb needed to overcome all obstacles. If all points are positive, then the delta will simply indicate 0, as the user does not need to climb to reach the waypoint.

$$\Delta h_{\text{climb}} = \max \left(0, -\min \left[\frac{d_i}{f} + h_{\text{terrain}}(i) - h_{\text{user}} \right] \right)$$

where d_i is the Haversine distance between the user and a distinct point, and $h_{\text{terrain}}(i)$ the terrain height at point i .

To be certain that every terrain datapoint between the user and the waypoint is evaluated, the points are separated equidistantly by the length of the tile's resolution. This is to ensure precision for the sake of the user's safety, at the cost of performance.

In addition to this, the algorithm will also return the glide altitude at the last point, situated at the waypoint, as it will allow us to add it to the minimum climb needed to gauge what the altitude will be at the waypoint when it is reached.

Tile controller

If the user decides to wander around in a very large radius, all the tile data cannot be loaded in the device's RAM and it would throttle performance with IO operations. In order to improve performance and use less memory, the tool should only load tiles which are in the path between, and included, the pilot and the waypoint.

To achieve this, we can make use of the LV95 grid identification of each tile and employ **Bresenham's line algorithm** [6] by treating each grid as a pixel. The algorithm uses integers to iteratively find pixels segmented by the line, by taking into account the error accumulation every time x is incremented. When the error is larger than the distance from the top of the pixel, then it increments y and resets the error for the new pixel. After the line has segmented from the start pixel to the end pixel, it will return the grid identification of the pertinent tiles. See Figure 5 for a visualisation and Annex A for the algorithm in depth.

In case the user flies to a neighboring tile in between tiles retrieval and the primary algorithm run, we add a buffer of one tile surrounding each intersected tile by the Bresenham algorithm, to prevent errors. We can be sure then that the algorithm always has elevation data to work with.

To optimize tile retrieval, we put the grid's identification and its elevation data in a map for $O(1)$ lookup. The Bresenham algorithm will tell the controller which tiles to add to the map, which ones to remove and which ones to keep, and this allows for efficient IO operations.

3.4 GPS

The GPS on the device will accurately return the user's latitude and longitude coordinates, but the altitude is not the altitude relative to the terrain. Rather, it returns the height above a WGS84 ellipsoid, which is a mathematical representation of the planet's shape. This means it's not true to life. The elevation above sea level is different, and can differ from this altitude.

Additionally, vertical positioning is harder for a GPS to estimate than lateral positioning.

4 Implementation

4.1 Technologies used

We opted to use Dart with the Flutter framework for its multi-platform capabilities, and it helps us by allowing us to test rapidly on a computer before deploying on a mobile device. Using the *geolocation* package, we manage permissions on devices and asynchronous position streams. The Dart library *image* is used to decode the GeoTIFF files. The elevation data is stocked in the red channel, so we have to extract the data byte by byte to construct our elevation data matrix. Another library is *http*, for REST API requests to fetch tiles. For the voice, we use *flutter_tts* to output a synthesized voice that reads the tuples returned by the 1D algorithm, and this allows the user to get the pertinent information hands-free, maintaining visual contact at all times with the surroundings.

4.2 Data optimisation

The image matrix is mapped to a Float32List matrix. It offers better performance than the default list in Dart, and uses less memory for the 4 million altitude points. This is important, as we will see that IO operations can be a major bottleneck in performance.

4.3 Parallel downloading

During testing, the downloading could sometimes get stuck on one tile, so to speed up the process by not being bound by a bottleneck, all tiles will be downloaded in parallel. The download speed is still bound by bandwidth, however. Additionally, files are saved according to their LV95 index and their bounding box (x-y-minLat-maxLat-minLong-maxLong.tif), as it allows us to search for the pertinent files without loading them first.

4.4 GPS altitude calibration

A critical challenge of the project is ensuring vertical accuracy. As seen in the conception phase, we need to calibrate the GPS' altitude readings to be more accurate to life. To do this, we ask the user to input their altitude at takeoff using an altimeter (a device that accurately measures the altitude) and to use the difference between that altitude and the ellipsoidal altitude to create an offset that is then applied continuously throughout the flight every time the GPS altitude is updated.

A better implementation would be to interface the user's variometer and get its altitude readings via Bluetooth, but this could not be done within the project's timeline.

4.5 User interface

To facilitate the handling of the tool before flight, we have implemented a UI:

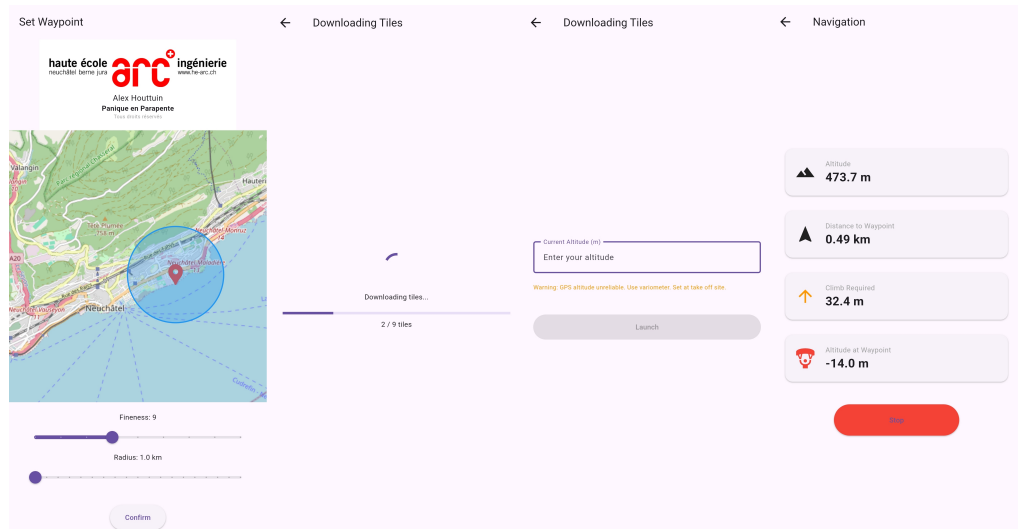


Figure 1: Screenshots of the UI in order of operation

Waypoint picker screen

The user can choose where the waypoint should be by tapping on the map at any desired location (within Switzerland). It is possible to choose the

fineness ratio and the wander radius.

Download screen

The necessary tiles determined by the coordinates and radius will be downloaded onto the device and the screen will indicate how many are left. When this is done, the user is asked to enter their altitude before takeoff using an altimeter, for calibration's sake.

Live screen

Upon pressing launch, the algorithm will run until the user closes the app. Normally, a pilot shouldn't look at the screen during flight but there are a few measurements to indicate the altitude, the distance to waypoint, the climb needed and the altitude at the waypoint.

5 Results and discussion

5.1 Topography

5.1.1 Tile download time

By assuming the user has, on average, the mean of Swisscom's 5G network as bandwidth at about 16.7 MB/s [7], we can plot a graph to show us the time to download all tiles within a given radius.

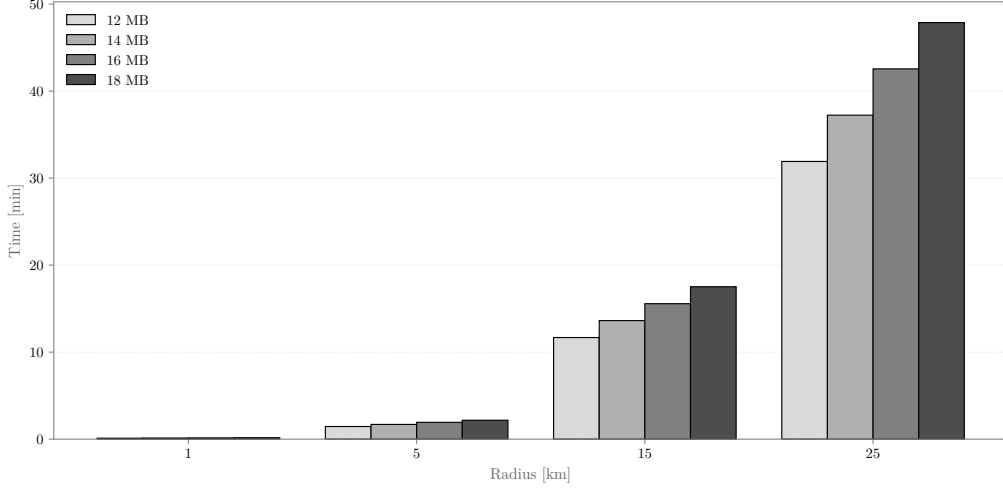


Figure 2: Tile download time relative to the radius

The time complexity is $O(r^2)$ due to the quadratic increase of the number of tiles in the bounding box defined by the radius (the square derived from the circle's surface $A = \pi r^2$). The time to download also increases drastically, as for a 5 km radius it takes about 1 minute to download all tiles, but for a radius of 25 km, it takes nearly 50 minutes. This confirms that real-time downloading of the full map is impossible "on the fly" during flight, and validates our primary objective to enforce offline as much as possible. The program prevents the user from taking flight before all tiles are downloaded.

5.1.2 Tile visualisation

To confirm that the tiles we get correspond to their actual coordinates, we do a side-by-side comparison using QGIS [8] to visualise a tile and the Swiss Confederation's geo map website at the given coordinates :

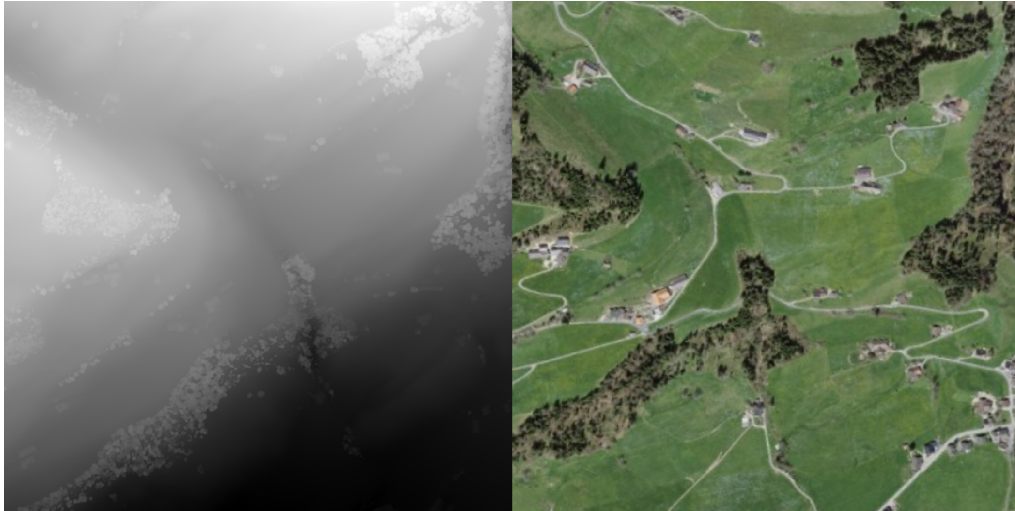


Figure 3: Side-by-side tile and screenshot from map.geo.admin.ch at coordinates 46.8303, 7.8518

We can observe that the tile matches real-life in Figure 3. We can also confirm that objects, such as trees and buildings, are indeed present, as the lighter the pixel is in brightness, the higher that point is in terms of elevation. This validates our use of this dataset, as it is based on a physical reality which helps the 1D algorithm in calculating clearance.

5.2 1D algorithm

We conduct a time analysis of the 1D algorithm and the tile controller to see if they fit into the 10-second tick interval.

5.2.1 Unit tests

For both the naive and the obstacle-aware algorithm, we've performed unit tests to ascertain their accuracy using a simulated matrix as elevation data. We validate the algorithms with three scenarios:

- Flat terrain, the user has to reach a point located 100 meters away at the same altitude. With a fineness ratio $f = 9$, the algorithm predicts the user needs to climb about 11 meters to reach that point in space.
- Major obstacle, a thin wall of 0.5 meters in width and at a very high altitude is situated between the user and the waypoint, the algorithm can detect the collision thus confirming its precision.
- Multiple obstacles, in this scenario, the user flies over a matrix with simulated ridges between 800 and 1200 meters while starting at a high altitude of 1500 meters. The test confirms that every point between the user and the waypoint is assessed, and is thus capable of handling irregular terrain.

5.2.2 Algorithmic time

For our needs, the algorithm needs to be fast and precise, to fit within the 10-second time tick. We have confirmed its precision, so we will now analyze its speed.

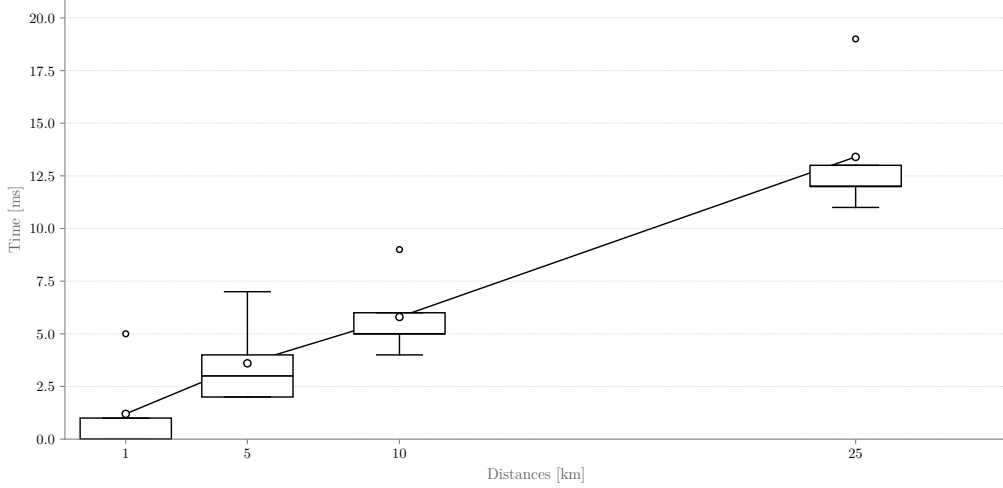


Figure 4: Boxplot for elapsed times per given distance

The linear nature, as shown in Figure 4, gives us the indication that the time complexity is $O(n)$. Since the algorithm does not cost much in time and is well under a second even for the 25 km distance. This is negligible compared to the 10-second tick interval. We can say, in this circumstance, that the algorithm does not need to be optimized further with parallelism, offloading to a server or some other complex optimization.

5.2.3 Tile controller

Here, we plot the Bresenham line algorithm to see if it is coherent, and we analyze the relationship of tile loading time and memory usage.

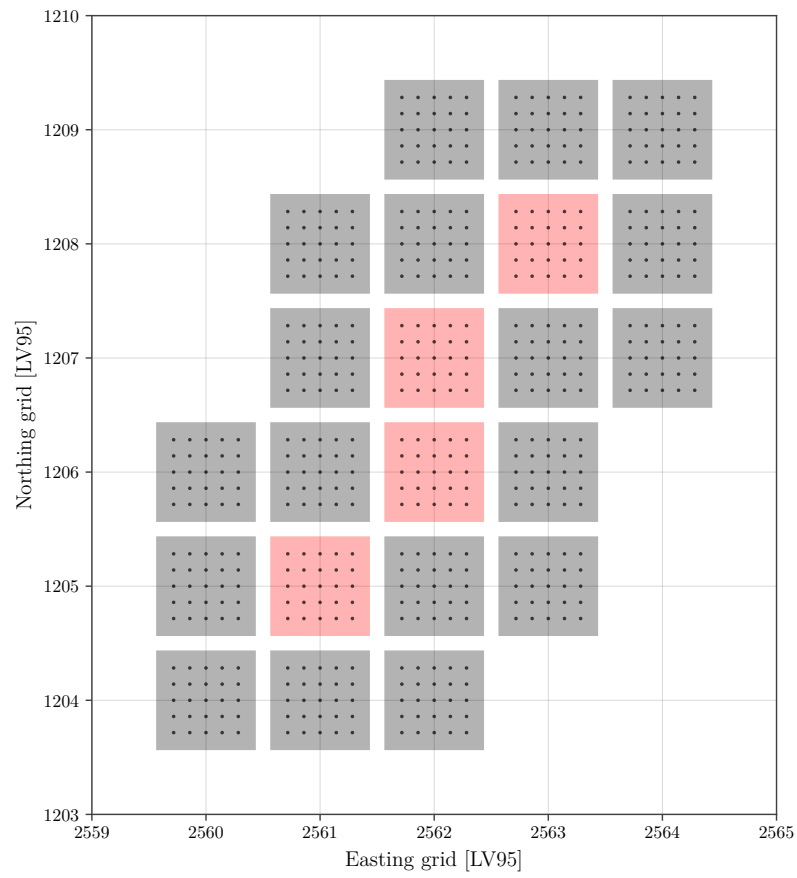


Figure 5: Bresenham algorithm tile loading

The Bresenham algorithm (as shown in Figure 5), works efficiently. The red tiles represent the LV95 grids segmented by the algorithm, while the grids surrounding it act as a buffer, in case the user wanders to a neighboring tile between the loading of the tiles and the execution of the primary algorithm. We can also deduce that, if the waypoint is horizontal or vertical relative to the user's position, it will intersect with less tiles than if the waypoint is on a diagonal path relative to the user. Mathematically, we can denote the latter as the maximum tile count $n = 5d + 9$, with d the distance in LV95 grid units (kilometers) between the user and the waypoint.

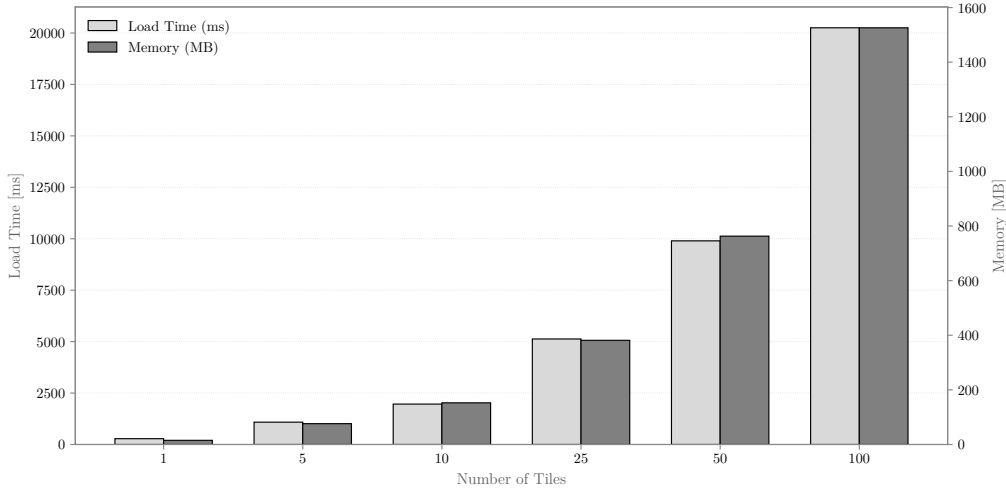


Figure 6: Time and memory tile loading

By analyzing both the loading time and the memory usage, as shown in Figure 6, we can observe a linear relationship between the tile count (n) and memory usage. Both time and space complexity are $O(n)$. Given the 10-second tick window, we determine that:

$$n < 50$$

Using the maximum tile count derived from the Bresenham algorithm with margin seen above ($n = 5d + 9$), we can solve for the maximum permissible distance (d):

$$50 > 9 + 5d$$

$$41 > 5d$$

$$d < \frac{41}{5} = 8.2 \text{ km}$$

Loading close to 50 tiles required for a 8.2 km path takes nearly the full 10-second window. However, this latency only applies for the very first run of the algorithm and the max permissible distance becomes much greater afterwards. Subsequent ticks only need to load an incremental difference (Δ tiles). The 8.2 km limit is thus only a constraint during takeoff, not continuous flight.

With these results, we have formulated a user guide in Annex E.

6 Conclusion

We have succeeded in implementing most of our primary objectives, and all of our secondary objectives of the project "Panique en Parapente". The tool is functional on mobile devices with limited resources and gives near-real-time feedback on the climb needed and the altitude at waypoint.

The 1D algorithm is both precise and fast (achieving $O(n)$), as it can detect a thin wall and it runs under a second for long ranges, easily fitting in the 10-second update interval. The tile controller solved the memory constraint by implementing a path-based map manager based on the Bresenham line algorithm. It caches the required maps and incrementally adds and removes them when needed.

While the ServiceElevation architecture was designed for abstraction, the current implementation remains bespoke to the SwissTopo STAC API due to the specific preprocessing required for this dataset. The abstraction layer for the ServiceElevation interface currently only supports the simulation implementation. Future work would involve standardizing and decoupling the SwissTopo-specific logic, to support both global data providers and other local providers.

6.1 Going further

6.1.1 Fast marching trees

While we could not implement the 2D algorithm, we have found an algorithm that could be interesting to implement as it works more efficiently than A^* in a 3D space called the Fast Marching Trees algorithm (abbreviated as FMT^*)

[9]. It is asymptotically optimal and uses a sample-based method of pathfinding. One issue with it is that, of course, it's more computationally expensive at $O(n \log n)$, and this is where HPC offloading could be interesting to implement, by sending the position of the user to the server and getting three outputs: the climb needed, the altitude at the waypoint and the heading of the path defined by the algorithm. The user would then need to follow the path following a path that has been made easier to follow by using Funnel's algorithm for example.

6.1.2 Hardware integration

Vertical accuracy needs to improve for the tool to be reliable. Future development could achieve this by integrating data from Bluetooth variometers. This would replace our barometric offset calibration and render flight both safer and easier for the user.

7 Sources

- [1] Fédération suisse de vol libre (FSVL), "Sécurité et Rapports d'accidents," shv-fsvl.ch. [Online]. Available: <https://www.shv-fsvl.ch/en/fluggebiete-sicherheit/safety/translate-to-englisch-unfallmeldungen/>. [Consulted on: 12 January 2026].
- [2] Office fédéral de topographie (swisstopo), "SwissSURFACE3D Raster," swisstopo.admin.ch. [Online]. Available: <https://www.swisstopo.admin.ch/fr/modele-altimetrique-swissurface3d-raster>. [Consulted on: 21 January 2026].
- [3] The Swiss Government, "SwissTopo STAC API," data.geo.admin.ch. [Online]. Available: <https://data.geo.admin.ch/api/stac/v0.9/collections/ch.swisstopo.swissurface3d>. [Consulted: 21 Jan 2026].
- [4] National Ocean Service (NOAA), "What is the distance between a degree of latitude and longitude?," oceanservice.noaa.gov. [Online]. Available: <https://oceanservice.noaa.gov/facts/longitude.html>. [Consulted on: 1 October 2025]

- [5] C. Veness, "Calculate distance, bearing and more between Latitude/Longitude points," Movable Type Scripts. [Online]. Available: <https://www.movable-type.co.uk/scripts/latlong.html>. [Consulted: 5 Nov 2025].
- [6] Wikipedia, "Bresenham's line algorithm," wikipedia.org, 19 Nov 2025. [Online]. Available: https://en.wikipedia.org/wiki/Bresenham%27s_line_algorithm. [Consulted: 20 Nov 2025].
- [7] Ookla, "Switzerland Speedtest Connectivity Report H1 2025", 2025. [Online]. Available: <https://www.ookla.com/research/reports/switzerland-speedtest-connectivity-report-h1-2025>. [Consulted on: 21 January 2026]
- [8] QGIS Development Team, "QGIS Geographic Information System," Open Source Geospatial Foundation Project, 2025.
- [9] L. Janson et al., "Fast Marching Tree: A Fast Marching Sampling-Based Method for Optimal Motion Planning in Many Dimensions," IJRR, vol. 34, no. 7, 2015.